

ChatVeritas Architecture

This document is the technical specification for ChatVeritas. It describes the repository as implemented: component boundaries, data contracts, artifacts, configuration, execution paths, and extension points. For a concise project introduction and launch commands, see [README.md](#).

1. Project Overview

ChatVeritas is a document-grounded Retrieval-Augmented Generation (RAG) application. It answers questions against a local corpus of UTF-8 text files instead of treating the language model as the source of project knowledge. The corpus is converted into embeddings and stored in a FAISS index; every question retrieves and reranks evidence before a response is generated.

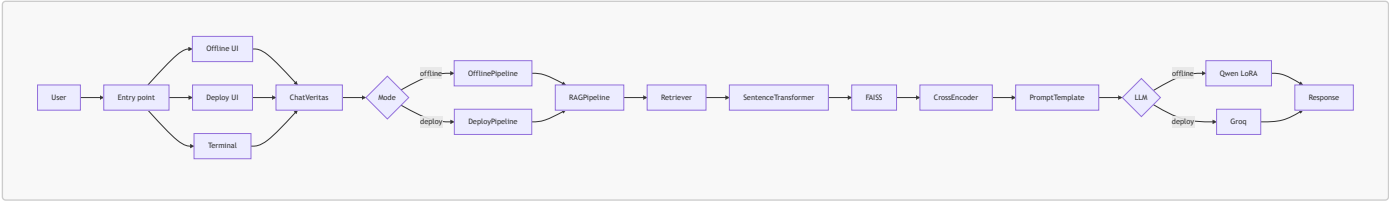
The application addresses two practical deployment cases with one retrieval system:

- **Offline mode** runs `Qwen/Qwen2.5-3B-Instruct` locally and can apply a PEFT LoRA adapter.
- **Deploy mode** uses the Groq OpenAI-compatible endpoint for generation when hosting the local model is impractical.

The design favors explicit ownership over opaque framework abstractions. `retrieval/` owns embeddings, FAISS search, and cross-encoder reranking; `llm/` owns backend-specific loading and generation; `prompts/` owns wording; and `pipelines/` connects those pieces. LangChain Core is used only for orchestration and prompt rendering, not to replace the retrieval algorithm or model backends.

2. High-Level Architecture

System view



All user-facing paths converge on `ChatVeritas.ask(question)`. The interface selects a mode-specific pipeline lazily; that pipeline uses the common LCEL chain in `RAGPipeline` and returns the same public result shape regardless of backend.

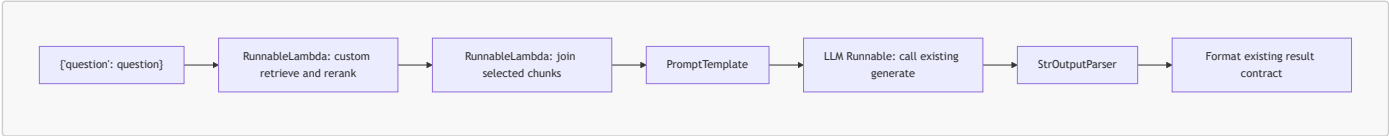
Runtime data contract

The only result contract exposed beyond `pipelines/` is:

```
{
  "response": str,
  "chunks": list[dict],
  "metrics": dict,
}
```

Each returned chunk contains `chunk`, `source`, `chunk_id`, `distance`, `faiss_rank`, and `rerank_score`. `metrics` combines retrieval timing and metadata with generation timing and prompt-token count. The Streamlit apps use this information to display the answer, evidence, sources, and diagnostic timings.

LCEL chain



`RunnablePassthrough.assign` retains the retrieval output while it adds `context`, `prompt`, `generation`, and `response`. This is important: the chain does not turn the retrieved chunks into generic LangChain documents or discard their project-specific metrics.

3. Repository Structure

Path	Why it exists	What belongs there	Primary users
<code>app_offline.py</code>	Starts the local web app.	Streamlit UI and resource caches for offline inference.	Local end users.
<code>app_deploy.py</code>	Starts the hosted-generation web app.	Streamlit UI and <code>.env</code> loading for Groq mode.	Deploy users.
<code>config/</code>	Separates settings from code.	<code>config.json</code> is active; <code>config_old.json</code> is not loaded by runtime code.	All pipelines and scripts.
<code>core/</code>	Holds application-wide concerns.	Config loading, logging, exceptions, and early thread limits.	Entry points and runtime modules.

Path	Why it exists	What belongs there	Primary users
<code>data/</code>	Holds user data and generated artifacts.	Raw documents, FAISS artifacts, processed JSONL, cache, and model offload data.	Ingestion, dataset preparation, retrieval, offline loading.
<code>interfaces/</code>	Preserves a stable application API.	The <code>ChatVeritas</code> facade.	Apps and terminal chat.
<code>llm/</code>	Encapsulates inference-provider behavior.	Base contract, LCEL adapter, local Qwen/LoRA, and Groq backend.	Pipelines.
<code>models/</code>	Separates trained artifacts from source code.	LoRA adapter, tokenizer files, and training checkpoints.	Fine-tuning and offline inference.
<code>pipelines/</code>	Owens execution order and backend selection.	Shared LCEL RAG pipeline plus offline/deploy specializations.	<code>ChatVeritas</code> .
<code>prompts/</code>	Gives prompt wording one owner.	The RAG prompt template and render helper.	<code>RAGPipeline</code> .
<code>retrieval/</code>	Keeps the retrieval strategy explicit and reusable.	FAISS retrieval and cross-encoder reranking.	<code>RAGPipeline</code> .
<code>scripts/</code>	Contains executable maintenance and lifecycle commands.	Ingestion, data preparation, training, terminal chat, inspection, scraping.	Developers/operators.
<code>tests/</code>	Captures behavioral contracts.	Unit tests for utilities, training validation, and LCEL orchestration.	Developers and CI.
<code>utils/</code>	Reusable fine-tuning support code.	Cache, Groq question generator, paragraph chunking utility.	Dataset preparation and tests.

The `__init__.py` files only mark importable application directories; they do not expose a second public architecture.

4. Module Documentation

This section documents every executable application module. Package marker files and tests are omitted because they contain no runtime implementation.

Entry points and interface

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
<code>app_offline.py</code>	Local Streamlit UI. <code>get_config()</code> caches config; <code>load_chatbot(use_lora)</code> caches <code>ChatVeritas</code> .	Reads configuration and the artifacts loaded by the offline pipeline; stores chat history in <code>st.session_state</code> ; displays response, chunks, sources, and timings.	Calls <code>apply_thread_limits</code> , <code>load_config</code> , and <code>ChatVeritas(mode='offline')</code> . It is launched by Streamlit, not imported by other application modules.	Reads model labels for the sidebar. It remains separate from <code>app_deploy.py</code> because LoRA selection and local loading are offline-specific.
<code>app_deploy.py</code>	Deploy Streamlit UI. <code>get_config()</code> and <code>load_chatbot()</code> mirror the offline entry point.	Loads <code>.env</code> , reads config, and displays the same result contract. It does not write project artifacts.	Calls <code>apply_thread_limits</code> , <code>load_dotenv</code> , <code>load_config</code> , and <code>ChatVeritas(mode='deploy')</code> .	Requires <code>GROQ_API_KEY</code> through <code>DeployLLM</code> . Keeping it separate keeps hosted credentials and deployment messaging out of local mode.
<code>scripts/chat.py</code>	Interactive local terminal chat. Public function: <code>main()</code> .	Prompts for LoRA use and user questions; prints answers; no persistent output.	Calls <code>apply_thread_limits</code> before constructing <code>ChatVeritas(mode='offline')</code> .	It intentionally contains no retrieval or model logic, making terminal behavior match the offline app.
<code>interfaces/chatveritas.py</code>	Defines the stable facade <code>ChatVeritas(mode, use_lora, config)</code> and <code>ask(question)</code> .	Accepts a non-empty question; returns the common result dictionary. Empty questions return empty response/chunks/metrics without constructing new state.	Called by both apps and terminal chat. Lazily imports <code>OfflinePipeline</code> or <code>DeployPipeline</code> ; calls their <code>run</code> .	Uses all runtime configuration indirectly. The facade isolates UIs from LCEL, retrieval, and LLM details and is the right place for future non-UI callers.

Core modules

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
<code>core/config.py</code>	<code>load_config()</code> is the single configuration loader.	Reads <code>config/config.json</code> ; returns a Python dictionary. It writes nothing.	Called by entry points, <code>ChatVeritas</code> when no config is injected, and lifecycle scripts.	Centralizing the project-root path prevents working-directory-dependent configuration behavior.
<code>core/constants.py</code>	Defines <code>THREAD_LIMIT_ENV_VARS</code> and <code>apply_thread_limits()</code> .	Sets environment defaults for OpenMP, MKL, OpenBLAS, NumExpr, tqdm, and Transformers verbosity.	Called before heavy imports by the apps and terminal chat.	Separate because import order matters: NumPy, FAISS, Sentence Transformers, and Transformers may initialize thread pools during import.
<code>core/logger.py</code>	<code>get_logger(name)</code> returns a configured named logger.	Writes formatted logs to stdout; avoids duplicate handlers for the same logger.	Used throughout runtime modules.	A centralized formatter makes instrumentation consistent and can later be extended with file or structured logging.
<code>core/exceptions.py</code>	Defines <code>ChatVeritasError</code> , <code>ModelLoadError</code> , <code>RetrievalError</code> , and <code>ConfigurationError</code> .	No I/O. These types communicate expected failure categories.	Raised by the interface, offline loader, and retriever.	Keeps caller-facing errors distinct from third-party exceptions.

Retrieval and prompt modules

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
<code>retrieval/retriever.py</code>	<code>Retriever(...)</code> loads the index, chunks, embedding model, and reranker. <code>retrieve(query)</code> executes two-stage retrieval.	Reads <code>index.faiss</code> and <code>chunks.pkl</code> . Returns <code>{'results': list[chunk], 'metrics': dict}</code> . It validates index/chunk count equality and rejects an empty index.	Created by <code>RAGPipeline</code> ; calls <code>SentenceTransformer</code> , <code>FAISS</code> , <code>NumPy</code> , and <code>Reranker.rerank</code> .	Uses <code>embedding</code> , <code>reranker</code> , <code>retrieval.top_k</code> , and <code>retrieval.faiss_candidates</code> . It is separate so index format, metrics, and retrieval policy remain independent of orchestration.
<code>retrieval/reranker.py</code>	<code>Reranker(model_name, device)</code> loads a <code>CrossEncoder</code> ; <code>rerank(query, candidates, top_k)</code> scores and sorts candidates.	Mutates candidates by adding <code>rerank_score</code> ; returns the highest-scoring <code>top_k</code> candidates. No files are written.	Constructed by <code>Retriever</code> ; called after FAISS candidate retrieval.	Uses <code>reranker.model</code> and <code>reranker.device</code> . It remains custom because it preserves the candidate metadata and ranking policy produced by <code>Retriever</code> .
<code>prompts/templates.py</code>	Holds <code>RAG_PROMPT_TEMPLATE</code> and <code>build_rag_prompt(question, context)</code> .	Accepts a question and joined context; returns the exact prompt string used by either backend.	Used by <code>RAGPipeline</code> ; depends on <code>LangChain Core PromptTemplate</code> .	There are no prompt-related config keys. The module separates wording from control flow and is the only place to edit the RAG instructions.

LLM modules

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
<code>llm/base.py</code>	Abstract <code>BaseLLM</code> defines <code>generate(prompt) -> (response, metrics)</code> and <code>as_runnable()</code> .	No I/O itself. <code>as_runnable()</code> preserves the same text-plus-metrics contract for LCEL.	Implemented by <code>OfflineLLM</code> and <code>DeployLLM</code> ; consumed by <code>RAGPipeline</code> .	The common contract makes backend replacement possible without changing the interface or result shape.
<code>llm/runnables.py</code>	Defines protocol <code>SupportsGeneration</code> and <code>create_generation_runnable(llm)</code> .	Wraps <code>generate</code> ; emits <code>{'response': str, 'metrics': dict}</code> .	Called through <code>BaseLLM.as_runnable()</code> by the pipeline.	It is separate so <code>LangChain</code> -specific adapter code does not leak into either model backend.
<code>llm/offline.py</code>	<code>OfflineLLM(config, use_lora)</code> loads and runs local Qwen. <code>generate(prompt)</code> applies the tokenizer chat template, calls <code>model.generate</code> , decodes only newly	Reads a local adapter if complete; otherwise resolves the configured Hugging Face adapter ID. May create	Created by <code>OfflinePipeline</code> ; uses <code>PyTorch</code> , <code>Transformers</code> , <code>PEFT</code> , and <code>BaseLLM</code> .	Uses <code>model</code> , <code>generation</code> , and optional <code>inference</code> settings. Separate helper methods make adapter

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
	generated tokens, and returns generation time and input-token count.	<code>data/model_offload/</code> . Does not alter model weights.		discovery, tokenizer fallback, memory-aware model loading, and LoRA application individually maintainable.
<code>llm/deploy.py</code>	<code>DeployLLM(config)</code> initializes an OpenAI client for Groq; <code>generate(prompt)</code> sends system and user messages to the configured model.	Reads <code>GROQ_API_KEY</code> from environment; returns response, wall-clock generation time, and provider prompt-token count. No files are written.	Created by <code>DeployPipeline</code> ; uses OpenAI client and <code>BaseLLM</code> .	Uses <code>generation.model</code> when present, otherwise <code>openai/gpt-oss-120b</code> , plus generation temperature and token limit. It remains custom to retain the Groq endpoint, system message, and metrics semantics.

Pipeline modules

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
<code>pipelines/rag_pipeline.py</code>	Abstract <code>RAGPipeline</code> owns <code>run(question)</code> and the common LCEL chain. Subclasses implement <code>_create_llm()</code> .	Reads retrieval artifacts through <code>Retriever</code> ; returns the common result contract. It writes no project artifacts.	Called by <code>ChatVeritas</code> through a concrete pipeline. It calls <code>Retriever.retrieve</code> , <code>PromptTemplate</code> , the LLM runnable, and <code>StrOutputParser</code> .	Uses retrieval, embedding, reranker, paths, and backend generation config. It is separate to eliminate duplicated offline/deploy orchestration without concealing custom retrieval.
<code>pipelines/offline_pipeline.py</code>	<code>OfflinePipeline(config, use_lora)</code> specializes <code>RAGPipeline</code> .	Creates the common retriever and an <code>OfflineLLM</code> ; uses the base <code>run</code> .	Built by <code>ChatVeritas</code> for offline mode.	Adds only the LoRA flag. Keeping it thin makes the difference from deploy mode explicit.
<code>pipelines/deploy_pipeline.py</code>	<code>DeployPipeline(config)</code> specializes <code>RAGPipeline</code> .	Creates the common retriever and a <code>DeployLLM</code> ; uses the base <code>run</code> .	Built by <code>ChatVeritas</code> for deploy mode.	Has no extra persistent artifacts. It differs only in generation backend.

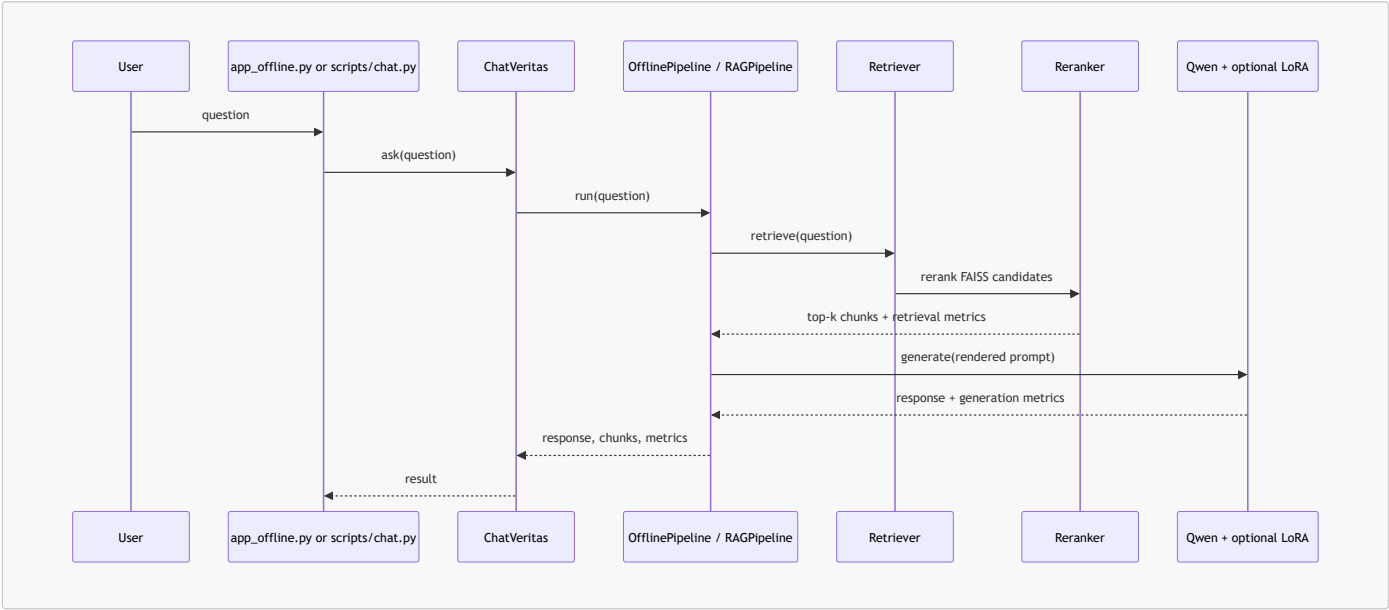
Data, training, and maintenance modules

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and extension rationale
<code>scripts/ingest.py</code>	<code>clean_text(text)</code> removes separator-only lines and extra blank lines. <code>main()</code> builds the vector store.	Reads <code>data/raw/*.txt</code> ; writes <code>data/vectorstore/index.faiss</code> and <code>data/vectorstore/chunks.pkl</code> . Each chunk stores <code>chunk_id</code> , <code>source</code> , and <code>chunk</code> .	Executed manually; runtime <code>Retriever</code> later consumes both artifacts. Uses <code>SentenceTransformer</code> , <code>FAISS</code> , <code>NumPy</code> , and <code>RecursiveCharacterTextSplitter</code> .	Uses <code>paths.raw_data</code> , <code>paths.vectorstore</code> , and <code>paths.retrieval.chunk</code> . It stays a script because of explicit offline lifecycle.
<code>scripts/prepare_finetune_dataset.py</code>	<code>main()</code> generates conversational training records from indexed chunks.	Reads <code>chunks.pkl</code> ; calls Groq through <code>QuestionGenerator</code> ; optionally reads/writes per-chunk cache JSON; writes JSONL to <code>paths.processed_data</code> .	Executed after ingestion; <code>fine_tune.py</code> consumes its output.	Uses <code>llm</code> , <code>cache</code> , and <code>paths</code> ; deliberately reads training examples for inference evidence.
<code>scripts/fine_tune.py</code>	Public helpers validate data/configuration and split by chunk; <code>main()</code> trains and saves a QLoRA adapter.	Reads processed JSONL; writes checkpoints, adapter weights, tokenizer, and <code>chatveritas_training.json</code> ; can upload to Hugging Face.	Executed manually; <code>OfflineLLM</code> consumes the saved adapter. Uses <code>Datasets</code> , <code>Transformers</code> , <code>PEFT</code> , <code>TRL</code> , <code>BitsAndBytes</code> , and Hugging Face Hub lazily.	Uses <code>model</code> , <code>paths</code> , and optional <code>hub</code> settings for fast validation and GPU loading.
<code>scripts/print_chunks.py</code>	<code>main()</code> prints indexed chunk metadata and text.	Reads <code>chunks.pkl</code> ; writes only console output.	Manual inspection tool after ingestion.	Uses <code>paths.vectorstore</code> ; separate avoids dependencies for retrieval.
<code>scripts/scrape_urls.py</code>	<code>sanitize_filename</code> , <code>scrape_url</code> , and <code>main()</code> fetch selected	Reads the hard-coded <code>URLS</code> ; writes <code>.txt</code> files under <code>data/raw/</code> .	Optional pre-ingestion helper; uses <code>Trafilatura</code> .	<code>URLS</code> and <code>OUTPUT_PATH</code> constants, not <code>config</code> .

Module	Purpose and public API	Inputs / outputs / artifacts	Dependencies and call graph	Configuration and
	URLs and save extracted Markdown-like text.			intentionally opt-in selection is editori
<code>utils/cache.py</code>	<code>CacheManager</code> provides SHA-256-keyed JSON <code>get_or_create</code> , plus read/write/count/clear helpers.	Reads/writes one JSON file per source chunk; uses atomic <code>os.replace</code> after a temporary file.	Created by dataset preparation when caching is enabled.	Cache location and from <code>cache</code> . Separ repeated paid API resuming interrupt
<code>utils/question_generator.py</code>	<code>QuestionGenerator</code> calls Groq to generate a topic and up to five validated questions for a paragraph/chunk.	Takes source text; returns <code>{'topic': str, 'questions': list[str]}</code> . It strips, deduplicates, limits, and validates response JSON.	Created by dataset preparation.	Uses <code>llm.model</code> , <code>llm.max_retries</code> RAG deployment t synthetic-data ger prompt and respo
<code>utils/paragraph_chunker.py</code>	<code>ParagraphChunker</code> token-counts with a Transformers tokenizer and splits paragraphs by sentence boundaries, with a word/token fallback for oversized sentences.	Takes raw text; returns <code>list[str]</code> ; no artifacts itself.	Covered by tests. It is not called by the current <code>scripts/ingest.py</code> , which uses <code>RecursiveCharacterTextSplitter</code> .	Constructor accep <code>max_tokens</code> (defa <code>min_paragraph_l</code> remains a reusable without silently ch ingestion format.

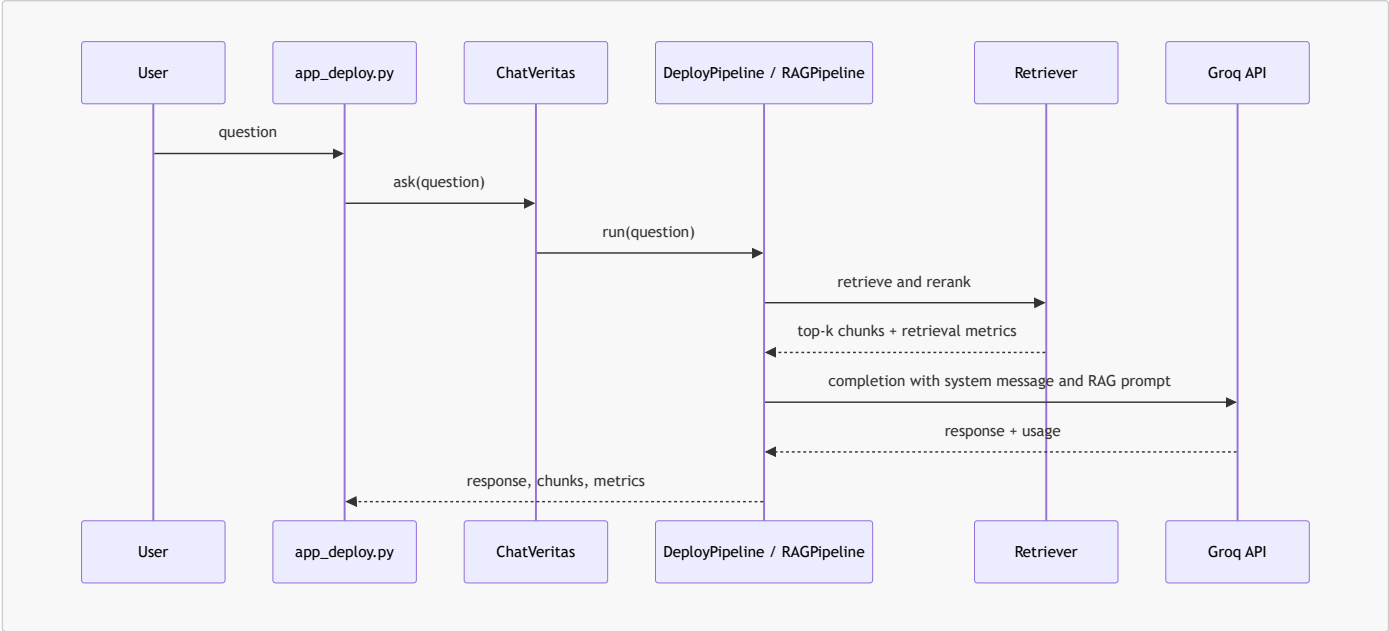
5. Runtime Pipelines

Offline pipeline



The offline Streamlit app applies thread limits before heavy imports, caches the configuration and chatbot resource, and exposes a LoRA toggle. The terminal script similarly applies limits but presents an interactive REPL. Neither alters retrieval or prompt construction.

Deploy pipeline

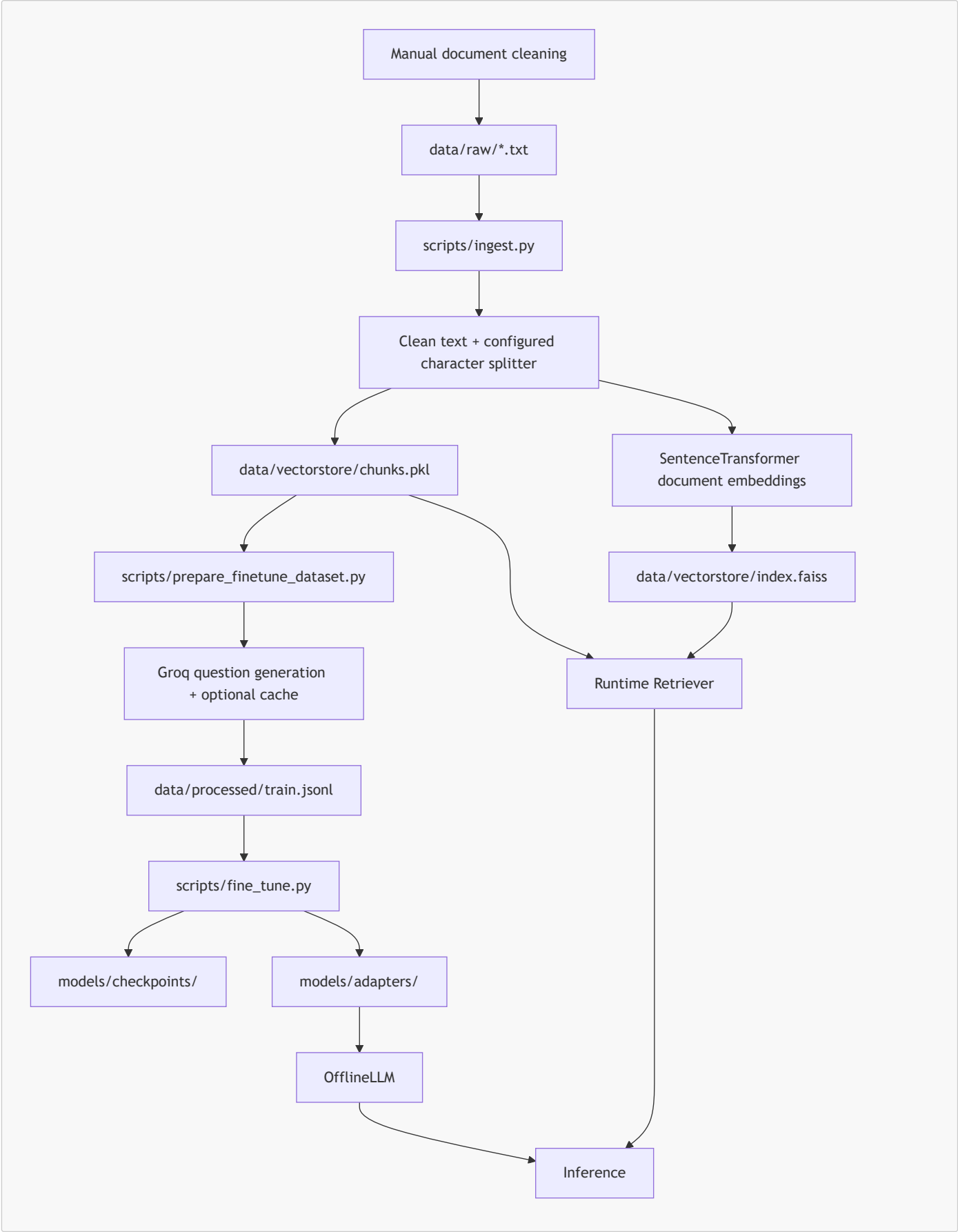


Deploy mode adds `.env` loading and requires `GROQ_API_KEY`; otherwise, its context and evidence path is identical to offline mode. This is why a response from either mode can be displayed by the same UI code.

Terminal pipeline

`scripts/chat.py` is not a separate retrieval pipeline. It asks the operator whether to use LoRA, creates the same offline `ChatVeritas` instance, then calls `ask` repeatedly until `exit`, EOF, or Ctrl+C. It does not maintain conversation memory: each question is an independent RAG invocation.

6. Data Pipeline



Stage	Producer	Artifact	Consumer	Notes
Source preparation	Developer or <code>scrape_urls.py</code>	<code>data/raw/*.txt</code>	<code>ingest.py</code>	Manual cleaning is intentionally outside code because corpus quality is a content decision.
Ingestion	<code>ingest.py</code>	<code>data/vectorstore/index.faiss</code>	Retriever	Flat L2 index containing float32 document

Stage	Producer	Artifact	Consumer	Notes
				embeddings.
Ingestion	<code>ingest.py</code>	<code>data/vectorstore/chunks.pkl</code>	<code>Retriever</code> , <code>prepare_finetune_dataset.py</code> , <code>print_chunks.py</code>	Pickled list mapping index position to chunk text, source, and ID.
Synthetic data	<code>prepare_finetune_dataset.py</code>	<code>data/cache/*.json</code>	Dataset preparation itself	Optional cache of validated Groq responses keyed by chunk hash.
Synthetic data	<code>prepare_finetune_dataset.py</code>	<code>data/processed/train.jsonl</code>	<code>fine_tune.py</code>	Conversational records with source/chunk provenance.
Training	<code>fine_tune.py</code>	<code>models/checkpoints/</code>	Training resume	Intermediate checkpoints; ignored by Git.
Training	<code>fine_tune.py</code>	<code>models/adapters/</code>	<code>OfflineLLM</code>	PEFT adapter, tokenizer, and training summary.

The active ingestion step uses `RecursiveCharacterTextSplitter` with paragraph, newline, sentence, word, and character fallbacks. The independent `ParagraphChunker` utility is token-aware and sentence-aware but is not wired into the current ingestion command; documentation must not conflate the two.

7. Retrieval System

Embedding and index compatibility

At ingestion, `SentenceTransformer(config['embedding']['model'])` encodes all chunk text. The array is cast to `float32`, added to `faiss.IndexFlatL2`, and written to disk. At query time, `Retriever` loads the same configured model, encodes the question, casts its vector to `float32`, and queries the index.

The document and query embedding models must remain compatible. Changing `embedding.model` requires rerunning ingestion; otherwise, the stored vectors and query vectors inhabit different spaces.

Two-stage ranking

- 1. **Candidate generation:** FAISS searches for `min(retrieval.faiss_candidates, index.ntotal)` nearest vectors by squared L2 distance. `IndexFlatL2` is a flat exhaustive index, not an approximate index despite the generic term “ANN” sometimes used in retrieval prose.
- 2. **Reranking:** `Reranker` creates `(question, chunk)` pairs for every FAISS candidate. Sentence Transformers `CrossEncoder.predict` scores all pairs, appends `rerank_score`, sorts descending, and returns `retrieval.top_k`.
- 3. **Context:** `RAGPipeline` joins only the final reranked chunk text with blank lines. It does not inject source names or scores into the LLM prompt. Sources and scores remain available to the UI through the result chunks.

The `Retriever` records embedding, FAISS, and reranking time separately. It also reports the actual candidate count, returned chunk count, average FAISS distance, and distinct source filenames.

Why the system is structured this way

Dense vector search makes broad candidate selection inexpensive for the local index. The cross-encoder then applies query-document attention to a much smaller set, improving the final ordering without scoring every corpus chunk. Keeping this logic in project code protects the exact metadata, timing, and ranking behavior that the UI and tests expect.

8. Fine-Tuning System

Dataset preparation

Dataset preparation intentionally starts from `chunks.pkl`, not a separately re-chunked corpus. For every chunk of at least 20 non-whitespace characters, `QuestionGenerator` asks Groq for one topic and up to five diverse questions that are answerable from that chunk. Each generated question produces a record whose assistant response is the original chunk text:

```
{
  "topic": "...",
  "source_file": "source.txt",
  "chunk_id": 12,
  "messages": [
    {"role": "user", "content": "Generated question"},
    {"role": "assistant", "content": "Original indexed chunk"}
  ]
}
```

`CacheManager.get_or_create` validates cached JSON before reusing it. Invalid, corrupt, or schema-incompatible cache entries are regenerated and overwritten.

Training

`scripts/fine_tune.py` first validates the JSONL schema, normalizes message whitespace, and groups examples by `source_file:chunk_id`. It splits whole groups into train and validation partitions so multiple generated questions for the same target chunk do not leak across partitions.

For a real training run, it requires CUDA, loads the Qwen base model in 4-bit NF4 mode, prepares it for k-bit training, and constructs a causal-LM LoRA configuration. `TRL SFTTrainer` receives only the `messages` field. The script saves checkpoints during training, then saves the final PEFT adapter and tokenizer to the configured adapter directory.

`--validate-only` is the supported fast check: it validates configuration and dataset content without importing the GPU training stack or loading a model. `--resume` optionally resumes the newest or named checkpoint.

Inference relationship

Offline inference checks whether `model.adapter_path` contains a complete adapter configuration and weight file. If not, it falls back to `model.adapter_repo_id`. If loading an adapter fails, it logs a warning and uses the base model instead. Fine-tuning changes generation behavior; RAG knowledge remains in FAISS and `chunks.pkl`.

9. LangChain Integration

LangChain Core was introduced as an orchestration refactor, not as a new RAG implementation. The application deliberately does **not** use `RetrievalQA`, `ConversationalRetrievalChain`, a LangChain vector store, or a built-in retriever abstraction.

Component	Location	Replaces	Why it improves the design
<code>PromptTemplate</code>	<code>prompts/templates.py</code>	Manual string formatting	Makes the existing fixed wording runnable-compatible and centrally represented without changing any instruction text.
<code>RunnableLambda</code>	<code>llm/runnables.py</code> , <code>pipelines/rag_pipeline.py</code>	Direct glue calls between stages	Adapts project contracts to LCEL while keeping the custom retriever and <code>generate</code> methods intact.
<code>RunnablePassthrough.assign</code>	<code>pipelines/rag_pipeline.py</code>	Local temporary pipeline variables	Builds an inspectable state containing retrieval output, context, prompt, generation, and response.
<code>StrOutputParser</code>	<code>pipelines/rag_pipeline.py</code>	Direct final-text pass-through	Makes the text-output step explicit in the LCEL sequence.

The custom components remain custom for specific reasons:

- **FAISS and retrieval strategy:** `Retriever` defines index validation, candidate count, chunk schema, metrics, and source attribution.
- **Cross-encoder reranker:** it works on the existing candidate dictionaries and adds `rerank_score` without converting the project to another document representation.
- **Ingestion and chunk handling:** the configured active splitter and the standalone paragraph-aware utility encode project-specific chunk semantics.
- **Prompt design:** only rendering changed; the wording remains owned by the project.
- **Offline and Groq backends:** custom adapters preserve LoRA discovery, memory controls, API endpoint details, system message, and metrics.

`scripts/ingest.py` already imported `langchain_text_splitters` before LCEL was added. That pre-existing use is retained because changing the active chunking implementation would alter index contents and is outside the orchestration refactor.

10. Configuration

The active configuration file is `config/config.json`. Paths are interpreted relative to the repository root when a caller resolves them. `config_old.json` is not read by current runtime code; it is a legacy reference and should not be treated as active configuration.

Active settings

Key	Current value	Controls	Consumers
<code>model.base_model</code>	<code>Qwen/Qwen2.5-3B-Instruct</code>	Base local causal language model.	<code>OfflineLLM</code> , <code>fine_tune.py</code> , offline UI label.
<code>model.adapter_repo_id</code>	<code>rahulboby/chatveritas-lora-adapter</code>	Hugging Face fallback for an incomplete local adapter; optional training upload target.	<code>OfflineLLM</code> , <code>fine_tune.py</code> , offline UI label.
<code>model.adapter_path</code>	<code>models/adapters</code>	Local output/input directory for the PEFT adapter.	<code>OfflineLLM</code> , <code>fine_tune.py</code> .
<code>embedding.model</code>	<code>sentence-transformers/all-MiniLM-L6-v2</code>	Document and query embedding model.	<code>ingest.py</code> , <code>RAGPipeline/Retriever</code> .
<code>embedding.device</code>	<code>cpu</code>	Device passed to SentenceTransformer.	<code>ingest.py</code> , <code>Retriever</code> .
<code>reranker.model</code>	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>	Cross-encoder reranker.	<code>RAGPipeline/Retriever</code> .
<code>reranker.device</code>	<code>cpu</code>	Device passed to CrossEncoder.	<code>RAGPipeline/Reranker</code> .
<code>retrieval.top_k</code>	<code>5</code>	Number of reranked chunks returned and used as context.	<code>Retriever</code> .
<code>retrieval.chunk_size</code>	<code>1000</code>	Character chunk size during active ingestion.	<code>ingest.py</code> .

Key	Current value	Controls	Consumers
<code>retrieval.chunk_overlap</code>	80	Character overlap during active ingestion.	<code>ingest.py</code> .
<code>retrieval.faiss_candidates</code>	30	Maximum FAISS candidates passed to the reranker.	<code>Retriever</code> .
<code>generation.max_new_tokens</code>	2048	Local generation limit and Groq <code>max_tokens</code> .	Both LLM backends.
<code>generation.temperature</code>	0.3	Enables/controls sampling for local and Groq generation.	Both LLM backends.
<code>paths.raw_data</code>	<code>data/raw</code>	Source corpus directory.	<code>ingest.py</code> .
<code>paths.vectorstore</code>	<code>data/vectorstore</code>	FAISS index and chunk metadata directory.	Ingestion, retriever, dataset preparation, chunk printing.
<code>paths.processed_data</code>	<code>data/processed/train.jsonl</code>	Generated conversational training dataset.	Dataset preparation, training.
<code>paths.training_checkpoints</code>	<code>models/checkpoints</code>	Training checkpoint directory.	<code>fine_tune.py</code> .
<code>dataset.max_chunk_tokens</code>	500	Present in active config but not read by the current dataset-preparation script.	Reserved/no current consumer.
<code>llm.provider</code>	groq	Allowed provider for synthetic question generation.	<code>prepare_finetune_dataset.py</code> .
<code>llm.model</code>	<code>openai/gpt-oss-120b</code>	Groq model for synthetic question generation.	<code>QuestionGenerator</code> .
<code>llm.temperature</code>	0.7	Sampling for synthetic question generation.	<code>QuestionGenerator</code> .
<code>llm.max_retries</code>	3	Maximum question-generation attempts.	<code>QuestionGenerator</code> .
<code>cache.enabled</code>	true	Enables on-disk synthetic-response cache.	Dataset preparation.
<code>cache.directory</code>	<code>data/cache</code>	Cache location.	<code>CacheManager</code> .
<code>training.epochs</code>	3	Number of SFT epochs.	<code>fine_tune.py</code> .
<code>training.batch_size</code>	2	Per-device training batch size.	<code>fine_tune.py</code> .
<code>training.gradient_accumulation</code>	8	Gradient accumulation steps.	<code>fine_tune.py</code> .
<code>training.learning_rate</code>	0.0002	Optimizer learning rate.	<code>fine_tune.py</code> .
<code>training.max_sequence_length</code>	2048	SFT maximum sequence length.	<code>fine_tune.py</code> .
<code>lora.r</code>	16	Low-rank adapter rank.	<code>fine_tune.py</code> .
<code>lora.alpha</code>	32	LoRA scaling factor.	<code>fine_tune.py</code> .
<code>lora.dropout</code>	0.05	LoRA dropout.	<code>fine_tune.py</code> .
<code>lora.target_modules</code>	Qwen projection module list	Modules adapted by LoRA.	<code>fine_tune.py</code> .

Optional keys accepted by code

Key	Code default	Effect
<code>generation.model</code>	<code>openai/gpt-oss-120b</code>	Groq deployment model. This is distinct from <code>llm.model</code> , which is for dataset preparation.
<code>inference.max_cpu_memory</code>	12GiB	Offline model CPU memory budget.
<code>inference.max_gpu_memory</code>	4GiB	Offline model GPU memory budget when CUDA is available.
<code>inference.offload_directory</code>	<code>data/model_offload</code>	Folder for Hugging Face model offload files.
<code>training.seed</code>	42	Grouped train/validation split seed and SFT data seed.
<code>training.validation_split</code>	0.05	Fraction of source-chunk groups held out for validation.
<code>training.eval_batch_size</code>	1	Per-device evaluation batch size.
<code>training.gradient_checkpointing</code>	true	Enables k-bit model preparation and trainer gradient checkpointing.
<code>training.packing</code>	false	Enables/disables SFT sequence packing.
<code>training.optimizer</code>	<code>paged_adamw_8bit</code>	Trainer optimizer identifier.
<code>training.lr_scheduler</code>	cosine	Learning-rate schedule.

Key	Code default	Effect
<code>training.warmup_ratio</code>	<code>0.03</code>	Trainer warmup proportion.
<code>training.weight_decay</code>	<code>0.0</code>	Trainer weight decay.
<code>training.max_grad_norm</code>	<code>1.0</code>	Gradient clipping norm.
<code>training.logging_steps</code>	<code>5</code>	Trainer logging interval.
<code>training.save_steps</code>	<code>50</code>	Checkpoint interval.
<code>training.eval_steps</code>	<code>50</code>	Validation interval when validation data exists.
<code>training.save_total_limit</code>	<code>2</code>	Number of checkpoints retained.
<code>lora.bias</code>	<code>none</code>	LoRA bias training policy.
<code>hub.private</code>	<code>false</code>	Visibility used when creating the optional adapter repository.

11. External Dependencies

Dependency	Purpose in this repository	Used in
PyTorch	Local tensor operations, inference mode, device/memory checks, CUDA cache handling.	<code>llm/offline.py</code> , training.
Transformers	Qwen tokenizer/model loading, chat template, generation, tokenizer-backed utility chunking.	Offline LLM, paragraph utility, training.
PEFT	Loads the inference LoRA adapter and defines QLoRA training configuration.	Offline LLM, training.
Sentence Transformers	Sentence embedding model and CrossEncoder reranker model.	Ingestion and retrieval.
FAISS	Stores and exhaustively searches float32 vectors with L2 distance.	Ingestion and retrieval.
NumPy	Ensures embedding/query arrays are <code>float32</code> and calculates average distances.	Ingestion and retrieval.
LangChain Core	<code>PromptTemplate</code> , LCEL runnables, passthrough assignment, and text output parser.	Prompts, LLM adapter, pipelines.
LangChain Text Splitters	Configured character splitter used by active ingestion.	<code>scripts/ingest.py</code> .
Streamlit	Chat UI, resource/data caching, expanders, metrics, and session history.	Both app entry points.
OpenAI client	OpenAI-compatible client configured with Groq base URL.	Deploy generation.
Groq SDK	Synthetic question-generation client.	<code>QuestionGenerator</code> .
TRL, Datasets, BitsAndBytes, Accelerate	Dataset construction, 4-bit QLoRA training, supervised fine-tuning, placement support.	<code>scripts/fine_tune.py</code> .
Hugging Face Hub	Optional adapter-repository creation and upload.	<code>scripts/fine_tune.py</code> .
Trafilatura	Optional URL extraction into source text files.	<code>scripts/scrape_urls.py</code> .
python-dotenv	Loads <code>GROQ_API_KEY</code> from <code>.env</code> .	Deploy app and dataset preparation.

12. Engineering Decisions

FAISS `IndexFlatL2`

The code uses a flat L2 index, which provides exhaustive nearest-neighbor search over the current vector store and keeps index behavior straightforward. The trade-off is memory and query cost at much larger corpus sizes; the code does not claim an approximate index or introduce compression.

Cross-encoder reranking

FAISS provides vector similarity candidates, while the CrossEncoder scores the question and full chunk jointly. The system performs the more expensive model scoring only for `faiss_candidates`, then keeps `top_k`. This preserves a distinct candidate-recall step and final relevance-ordering step.

LoRA / QLoRA

The project trains a small PEFT adapter rather than full model weights. That keeps the base model external and makes the trained artifact portable. Training uses 4-bit loading and k-bit preparation to reduce GPU memory requirements.

Chunking and corpus alignment

The active ingestion splitter prioritizes paragraph/newline/sentence boundaries while operating with character limits. Dataset generation then reads the saved chunks rather than recreating them. This prevents the fine-tuning answer text from drifting away from the retrieval corpus.

Streamlit plus terminal entry points

Streamlit provides a practical local web interface with cached heavyweight resources and evidence inspection. The terminal script provides a minimal interactive option without UI state. Both use `ChatVeritas`, so they do not fork application behavior.

Groq deployment backend

Hosted generation makes deployment feasible where a Qwen 3B model and adapter cannot be hosted within available resources. Retrieval remains local/project controlled, so changing the LLM backend does not change source selection.

LangChain Core, not a prebuilt RAG chain

LCEL improves composition, traceability, and separation of pipeline state. Prebuilt retrieval chains would hide the index, reranking, prompt, and metrics contracts that are the central engineering decisions of this application.

Modular application architecture

The directory boundaries map to responsibilities rather than framework types. They permit independent evolution of the retriever, prompt, model backend, and entry points while preserving one public `ChatVeritas` interface.

13. Project Lifecycle

The following is the full developer workflow from an empty local corpus.

1. **Install the environment.** Install `requirements.txt`; configure a `GROQ_API_KEY` for dataset generation and deploy mode.
2. **Prepare the corpus.** Manually clean content and store `.txt` files in `data/raw/`. Optionally edit `scripts/scrape_urls.py` and run it to create source files from selected URLs.
3. **Build retrieval artifacts.** Run `python scripts/ingest.py`. Confirm `index.faiss` and `chunks.pkl` are generated in `data/vectorstore/`.
4. **Inspect the corpus if needed.** Run `python scripts/print_chunks.py` to check source attribution, identifiers, and chunk text.
5. **Create training records.** Run `python scripts/prepare_finetune_dataset.py`. This requires the vector store and Groq credentials and produces `data/processed/train.jsonl`.
6. **Validate before training.** Run `python scripts/fine_tune.py --validate-only`. Correct data/configuration failures before allocating GPU resources.
7. **Train or resume.** Run `python scripts/fine_tune.py`, or add `--resume` to use the latest checkpoint. Expect checkpoints and a completed adapter.
8. **Run inference.** Use `app_offline.py` or terminal chat for local Qwen, optionally with the adapter; use `app_deploy.py` for Groq generation.
9. **Iterate safely.** Any embedding-model or chunking change requires a new vector store. Any base-model change requires a compatible newly trained adapter. Prompt changes require no artifact regeneration but affect generation behavior.

14. Future Roadmap

No explicit future-version roadmap, TODO list, issue reference, or planned feature list exists in the current source documentation. Accordingly, this document does not assert plans for hybrid retrieval, BM25, reciprocal-rank fusion, conversation memory, evaluation tooling, semantic chunking, or benchmarking. Those would be new proposals rather than documented repository plans.

15. Developer Notes

Common extension points

- **Prompt changes:** Edit only `prompts/templates.py`. Preserve `{context}` and `{question}` unless `RAGPipeline._prompt_inputs` is changed in tandem.
- **New LLM backend:** Implement `BaseLLM.generate(prompt)`, return the same `(response, metrics)` tuple, and add a thin `RAGPipeline` specialization. `as_runnable()` then integrates it with LCEL automatically.
- **Embedding model:** Change `embedding.model` and rerun ingestion before using runtime retrieval. Do not mix an index built with one model and queries embedded by another.
- **Reranker:** Change `reranker.model` or device for a compatible CrossEncoder. To alter ranking behavior, update `Reranker.rerank` while retaining the candidate dictionary contract expected by the UI.
- **Retrieval policy:** Adjust `top_k` or `faiss_candidates` first. Changes to the index type or chunk schema require coordinated changes in ingestion and retrieval.
- **Training:** Keep LoRA target modules compatible with the selected base architecture. Use a new adapter directory when changing base models.

Debugging and observability

- Start with the Streamlit **RAG Metrics** and **Retrieved Context** expanders. They expose retrieved chunks, FAISS distance, cross-encoder score, sources, and timing metrics.
- Logs are emitted to stdout in a consistent module-qualified format. Use the logger names to isolate retrieval, pipeline, or backend failures.
- A missing or out-of-sync vector store fails during `Retriever` construction; rerun ingestion instead of bypassing its count validation.
- Missing `GROQ_API_KEY` fails early in deploy generation and dataset preparation. Verify `.env` loading and environment scope.
- For local loading problems, inspect adapter completeness and the configured base model; `OfflineLLM` logs whether it chose a local adapter, Hub fallback, or base-model fallback.

Performance profiling

The result metrics are the first profiling surface:

- `embedding_time_ms` identifies query-embedding cost.
- `retrieval_time_ms` identifies FAISS search cost.
- `reranking_time_ms` identifies CrossEncoder cost.
- `generation_time` identifies LLM latency.
- `prompt_tokens`, `faiss_candidates`, and `retrieved_chunks` explain the amount of work at each stage.

For controlled comparisons, change one configuration value at a time, retain the same corpus/index, and record the returned metric dictionary alongside answer quality observations.